fk_cascade_probe — guide

Revision: 1 **Last modified:** 2026-09-26T13:02:00Z

Source: `constitution/scripts/codegraph/fk_cascade_probe.py` (288 lines). Citations below are `fk_cascade_probe.py:<line>`. Tests: `scripts/codegraph/tests/test_fk_cascade_probe.py` (42 tests). Paired mutation harness: `scripts/codegraph/tests/test_fk_cascade_probe_mutations.sh` (golden + 2 controls + 45 mutants).

Overview

A read-only decision tool for ONE installed CodeGraph build: can its bulk-index windows fall into the FK-cascade full-scan trap? The trap (header lines 8-21): `nodes.id` is a primary key and the store path writes `INSERT OR REPLACE INTO nodes`, so a duplicate node id is a DELETE + INSERT; `edges.source`, `edges.target` and `unresolved_refs.from_node_id` are `FOREIGN KEY ... REFERENCES nodes(id) ON DELETE CASCADE` and the store runs with `PRAGMA foreign_keys = ON`; during a fresh bulk index the indexer DROPs secondary indexes, and if the only index serving a child FK column is among them, every duplicate-id REPLACE becomes a FULL SCAN of the child table. The header quotes "2,223x slower at 1/6 scale" and a real 584,601-file run that stalled at 52% (lines 20-21); those two figures are quoted from the header and were not re-measured for this guide (the measured comparison in **Last verified** below is a separate, single-run observation).

Two independent layers; exit 0 requires BOTH to agree with "safe" (header lines 23-30):

- **STATIC** (lines 127-138): parse the target's own `db/schema.sql` and `db/index.js`, derive every FK child column of `nodes(id)`, and for each bulk window check that at least one index with that column LEFTMOST survives the window (a never-dropped UNIQUE index counts).

- **MEASURED** (lines 197-236): build a scaled scratch DB from the target's own schema, recreate exactly the indexes that survive each window, and time `INSERT OR REPLACE` of an EXISTING node id with `foreign_keys=ON`.

It never opens, reads, or writes a live `.codegraph/` index and never runs the CodeGraph binary: its inputs are two text files of the target and its only writes are one scratch SQLite DB (line 198) plus the optional evidence file.

It is also a hard gate. `codegraph_safe.sh` (the sanctioned single-writer entry) runs `fk_cascade_probe.py --work-dir <runs>/probe --json-out <runs>/probe_verdict.json` when the patch tool reports "patch not needed", and REFUSES to index with stock CodeGraph unless the probe exits 0 (`codegraph_safe.sh:469-471` ; documented in `codegraph_safe.md`). Exit 1 (HAZARD) and exit 2 (CANNOT_EVALUATE) therefore both block a bulk index.

Prerequisites

- `python3` , standard library only (imports lines 58-67).
- Default target resolution needs `npm` on `PATH` (`npm root -g` , line 83) and a global `@colbymchenry/codegraph` install; pass `--dist` to skip both.
- Scratch space for the measured layer: the header states ~0.5 GB at the default scale (line 51). Observed on this host (tmpfs work dir): default-scale run 16-18 s wall clock, max resident set ~96 MB, work dir left empty afterwards.
- A DB library capable of `PRAGMA foreign_keys` (Python's bundled `sqlite3`).

Usage

```
python3 scripts/codegraph/fk_cascade_probe.py [--dist <codegraph lib/dist dir>] [--threshold-s 0.05]
    [--scale-nodes N] [--scale-refs N] [--scale-edges N] [--static-only] [--json-out FILE] [--work-dir DIR]
```

Flag	Meaning	Source
`--dist DIR`	The target's `lib/dist` directory (it must contain `db/schema.sql` and `db/index.js`). Default: the first sorted platform package under `$(npm root -g)/@colbymchenry/codegraph/node_modules/@colbymchenry/` that has `lib/dist/db/index.js`. Given, `npm` is never consulted.	lines 81-93, 241, 252
`--threshold-s S`	A measured window is a hazard when its median REPLACE time is strictly greater than `S` seconds. Default `0.05`.	lines 229-230, 242
`--scale-nodes / --scale-refs / --scale-edges N`	Rows in the scaled scratch DB. Defaults 1,000,000 / 4,000,000 / 1,200,000.	lines 243-245
`--static-only`	Skip the measured layer entirely (no scratch DB or temp dir is created; the document has no `measured` / `scale` keys).	lines 246, 261
`--json-out FILE`	Also write the stdout document to `FILE` (identical bytes), including when the verdict is CANNOT_EVALUATE.	lines 247, 279-283
`--work-dir DIR`	Put the scratch DB `fk_probe.db` in `DIR` (must already exist). `DIR` is never deleted; the DB inside is removed at the end of a successful measured run. Default: a fresh `cg_fk_probe_*` dir under `$TMPDIR`, removed afterwards.	lines 248, 198, 235, 262-267
`-h`, `--help`	argparse help.	line 240

Exit codes

Code	Verdict	Meaning
0	`SAFE`	static AND measured agree there is no hazard (lines 271-272)
1	`HAZARD`	a window leaves an FK child column unserved and/or a measured REPLACE exceeds the threshold
2	`CAN-NOT_E-VALUATE`	missing/unparseable target, unexpected schema shape, disk/DB error, or ANY unexpected exception — FAIL-CLOSED, never reported as safe (lines 273-278; §11.4.201(4))

argparse usage errors (unknown flag, non-numeric value) also exit 2 but print NO JSON document on stdout.

Output

One JSON document on stdout (`json.dumps(doc, indent=2)` , lines 279-283). Keys: `probe` (`"codegraph-fk-cascade"`), `verdict` , `dist` , `fk_children_of_nodes` (table/column pairs found), `bulk_lists` (the three `BULK_*_NAMES` lists plus any other `BULK_..._NAMES` list found, echoed as evidence), `static` (per window `parse` / `ref` / `edge` : `served` , `unserved` , `hazard`), and unless `--static-only` : `measured` (per window `parse` / `ref` : `indexes_present_for_fk` , `replace_existing_s` [3 timings], `median_s` , `hazard`) and `scale` . On exit 2 the document carries an `error` string instead of results. stderr is silent on every evaluated run.

Examples (each was run; output observed on 2026-09-26)

1. Default target, static only — the stock CodeGraph 1.6.0 platform package on this host:

```
python3 scripts/codegraph/fk_cascade_probe.py --static-only ; echo rc=$?
```

Observed: `rc=1`, `verdict: HAZARD`, `dist` resolved to `.../@colbymchenry/codegraph/node_modules/@colbymchenry/codegraph-linux-x64/lib/dist`. Both the `parse` and `ref` windows list `edges.target` and `unresolved_refs.from_node_id` as unserved (candidates `idx_edges_target_kind`, `idx_unresolved_from_node`, `idx_unresolved_from_name` are all dropped in those windows); `edges.source` stays served by the never-dropped UNIQUE `idx_edges_identity`. The `edge` window alone lists only `edges.target`.

2. The same check against the patched private runner (`fkidx1`) — static only:

```
python3 scripts/codegraph/fk_cascade_probe.py \
  --dist "$HOME/.cache/helix/codegraph_runner/1.6.0-fkidx1/lib/dist" --static-only ; echo rc=$?
```

Observed: `rc=0`, `verdict: SAFE`; all three windows report no unserved column (`edges.source` via `idx_edges_identity`, `edges.target` via `idx_edges_target_kind`, `unresolved_refs.from_node_id` via `idx_unresolved_from_node`). The patched runner's `BULK_PARSE_INDEX_NAMES` has 14 entries against the stock 15.

3. Full default-scale run, stock vs patched (the measured layer), scratch DB in a work dir:

```
mkdir -p /path/to/scratch
python3 scripts/codegraph/fk_cascade_probe.py --work-dir /path/to/scratch --json-out /path/to/
        verdict.json ; echo rc=$?
python3 scripts/codegraph/fk_cascade_probe.py --dist "$HOME/.cache/helix/codegraph_runner/
        1.6.0-fkidx1/lib/dist" \
  --work-dir /path/to/scratch --json-out /path/to/verdict_patched.json ; echo rc=$?
```

Observed (single run each): stock `rc=1`, `HAZARD`, median REPLACE `0.603311 s` (parse) / `0.650082 s` (ref), both above the 0.05 s threshold, `indexes_present_for_fk` only `["idx_edges_identity"]`, 17.5 s wall clock; patched `rc=0`, `SAFE`, median `0.000224 s` (parse) / `0.000127 s` (ref), `indexes_present_for_fk` = `idx_edges_identity`, `idx_edges_target_kind`, `idx_unresolved_from_node`, 16.4 s wall clock. The scratch directory was empty

afterwards in both runs (`ls -A` count 0). This gives a stock-to-patched median ratio of about 2,700x (parse) and 5,100x (ref) on one run — an observation, not a benchmark.

4. Small scale, evidence file written — `--json-out` is byte-identical to stdout:

```
python3 scripts/codegraph/fk_cascade_probe.py --scale-nodes 20000 --scale-refs 80000 --scale-edges 24000 \
  --json-out evidence.json --work-dir wd > stdout.json ; echo rc=$? ; cmp stdout.json evidence.json && echo identical
```

Observed against the stock dist: `rc=1` (the static layer already fails), `identical`, measured medians `0.0105 s` (parse) and `0.0103 s` (ref) — BELOW the 0.05 s threshold, so both measured `hazard` flags were `false`; only the static layer made the verdict HAZARD. Small scales under-report the measured hazard; use the default scale (example 3) for a measured verdict you intend to rely on.

5. An unreadable target — fail-closed:

```
python3 scripts/codegraph/fk_cascade_probe.py --dist /nonexistent/dist --static-only ; echo rc=$?
```

Observed: `rc=2`; stdout is `{"probe": "codegraph-fk-cascade", "verdict": "CANNOT_EVALUATE", "dist": "/nonexistent/dist", "error": "cannot read target files: [Errno 2] No such file or directory: '/nonexistent/dist/db/schema.sql'"}` (pretty printed).

6. Running the tests and the mutation harness:

```
python3 scripts/codegraph/tests/test_fk_cascade_probe.py          # 42 tests, ~3 s
bash    scripts/codegraph/tests/test_fk_cascade_probe_mutations.sh # ~2.5 min; exit 0 only if every mutant is killed
```

Observed: `Ran 42 tests ... OK`; the harness printed `GOLDEN PASS`, `CONTROL unmodified-copy PASS`, `CONTROL dead-variant KILLED`, then `MUTANT M01..M45 KILLED`, `REAL-TOOL sha256 unchanged`, `TALLY mutants killed=45 of 45`, `ALL MUTANTS KILLED`. Set `TOOL=<path>` to point the test suite at a different copy of the probe (the harness does this for each mutant).

Edge cases and guarantees

Fail-closed inputs (each has a test; all exit 2 with `verdict: CANNOT_EVALUATE`):

Condition	`error` text starts/contains	Source
`schema.sql` or `index.js` unreadable	`cannot read target files`	lines 98-102
A `BULK_PARSE/REF/EDGE_INDEX_NAMES` list absent or empty	`list <NAME> not found/empty (upstream shape changed)`	lines 104-106
No `FOREIGN KEY ... REFERENCES nodes(id)` in `schema.sql`	`no FOREIGN KEY ... REFERENCES nodes(id) found`	lines 111-112
`npm` not on `PATH` / exits non-zero (no `--dist`)	`npm root -g failed` (npm's own stderr passes through)	lines 82-85
No `@colbymchenry/codegraph` install under the npm root	`no @colbymchenry/codegraph install under`	lines 86-88
No platform package carries `lib/dist/db/index.js`	`no platform package with lib/dist/db/index.js`	line 93
Measured layer: a NOT NULL column without default, not primary key, unknown to the probe	`<table>.<col> is NOT NULL without default and unknown to the probe (schema changed)`	lines 152-165
Measured layer: `nodes` / `unresolved_refs` / `edges` missing after the schema loads	`table <name> missing after schema load`	lines 159-162
ANY other exception (bad schema SQL, unusable `--workdir`, disk error)	`<ExceptionName>: <message>`, e.g. `OperationalError: ...`	lines 276-278

The shape guard exempts columns that have a default, columns that are primary keys, and the columns the probe fills. `--static-only` never builds a DB, so a schema the measured layer rejects can still get a static verdict.

Static-layer rules (all tested; lines 69-74, 108-116, 127-138):

- An index serves an FK column only when it is on the FK-child table AND the column is its LEFTMOST column; a trailing order suffix (`from_node_id DESC`) is stripped; `FOREIGN KEY` / `CREATE INDEX` matching is case-insensitive and whitespace-tolerant (`CREATE TABLE IF NOT EXISTS` must be uppercase).
- Windows (lines 119-123): `parse` = parse list + edge list, `ref` = ref list + edge list, `edge` = edge list. The parse and ref windows both enter the edge window, so an edge-list drop of the only index for a column is a hazard in all three. The verdict uses `parse` and `ref` only (line 260); `edge` is informational.
- A `BULK_..._NAMES` list other than the three named windows is echoed under `bulk_lists` but never used as a window.
- The regexes run over the raw file text. **Observed:** a commented-out line (`-- CREATE INDEX IF NOT EXISTS idx_unresolved_from_node ON unresolved_refs(from_node_id);`) was still counted as a serving index, and a commented `FOREIGN KEY` clause is still counted as an FK (which is why the fail-closed "no FK found" test edits the referenced table rather than commenting the clause out). CodeGraph's real schema does not comment out such lines today; a schema that did would be misjudged (a possible false SAFE for a commented-out index).

Measured-layer behaviour:

- The scaled DB is built from the target's own `schema.sql` (line 145); every trigger is dropped first (lines 146-147, the bulk window has no FTS sync triggers) and every non-UNIQUE index is dropped (lines 148-150; UNIQUE indexes stay, as in the real window). Only `parse` and `ref` are measured (line 209).
- Per window, the first surviving index of each served FK column is recreated from the schema's own DDL (lines 212-220); three existing node ids (10, `scale-nodes // 2` , `scale-nodes - 100`) are REPLACEd with `foreign_keys=ON` and committed, timed with `perf_counter` (lines 208, 222-227); `median_s` is the middle of the three (line 228).

- **Keep `--scale-nodes` above 100.** The third pick is `scale-nodes - 100`; below 100 that is a node id the scratch DB never generated (source-derived; the run itself still completes — observed with `--scale-nodes 50`), so that REPLACE is a plain insert with nothing to cascade and the measured layer says little.
- **Observed:** if the measured layer fails while `--work-dir` is used (exit 2, e.g. the shape guard), the partial `fk_probe.db` is left behind in that directory; only the no-`--work-dir` temp dir is always removed (line 267 runs in a `finally`). Remove a leftover `fk_probe.db` by hand; the next run replaces a stale one (lines 142-143).
- Timing is machine-dependent; the tests therefore assert only RATIOS for the measured layer (a hazard window at least 10x a same-run indexed window, and at least 10x a safe target's window; observed 61-325x at that scale) and use `--threshold-s -1` / `1000` to force each measured verdict deterministically.

Internal behaviour

`main()` (lines 239-284) builds one result document and maps it to an exit code: `find_default_dist` unless `--dist` (252) -> `load_target` reads `schema.sql` / `index.js` and derives FK children, index definitions and the three windows (254) -> `static_layer` (257) -> unless `--static-only`, `measured_layer` in a scratch dir (261-270) -> verdict `HAZARD` when the static parse/ref windows or any measured window is a hazard (260, 268, 271-272). The two layers control each other: a static "unserved" column predicts which index is absent from the measured window, and a measured slowdown independently corroborates or contradicts it (header line 273-274 cross-reference to §11.4.273).

Related scripts

- `codegraph_safe.sh` — the sanctioned single-writer entry; calls this probe when no runner patch is needed and refuses to index unless it exits 0 (`codegraph_safe.md`).
- `fk_index_patch.py` and `runner_patches/fkidx1.py` — build the private patched runner this probe is the oracle for (`fk_index_patch.py` header).

- `index_watch.py` — the progress-proven watcher for the bulk run this probe protects (`index_watch.md`).
- `owed_report.py` — lists which governed tools still lack tests or a guide (`owed_report.md`).
- constitution §11.4.78 / §11.4.80 (CodeGraph), §11.4.115 (RED on the broken artifact), §11.4.201(4) (fail closed), §11.4.273 (control needles), §1.1 (paired mutation), §11.4.18 (this guide).

Last verified

2026-09-26, against the tool at sha256 `8e830baec137823d5aba7e9fe356a79c309f7abcc7c3044de063b1be033b0e02` (unchanged from `git HEAD` of the constitution submodule): `test_fk_cascade_probe.py` 42/42 GREEN across repeated runs; mutation harness golden PASS, unmodified-copy control PASS, dead-variant control KILLED, 45/45 mutants KILLED across three consecutive runs, real tool byte-identical afterwards; timing-based mutants (`insert or replace` , `foreign_keys=ON` , index recreation) killed 5/5 each at host load average ~44. Examples 1-5 above were run on CodeGraph 1.6.0 (stock platform package and the private `1.6.0-fkidx1` runner). Not covered: the measured layer at the default scale is exercised by examples 3 only (not by the test suite, which uses scales up to 10,000 nodes / 150,000 refs), and dropping the `UNIQUE` -index keep in the scratch DB (lines 148-150) is not observable from the probe's output, so no mutant targets it.